Agentic development Reference Lab execution

Reference Lab execution report • Software Engineering 2026–2027

Student account: Yvesch12
Tool: OpenAI Codex in the Windows / VS Code development workspace
Reference project: Library Management REST API

Purpose and scope

This report documents the controlled Reference Lab, rather than a separate final-project idea. It summarizes how instructions, approval boundaries, generated artifacts, failures, tests, and refinements were used across the eight reference labs. It does not reproduce the full source code.

Implementation and evidence location

https://github.com/Yvesch12/library-api/tree/main
Completed implementation checkpoint: tag lab-8, commit `cabb823`.
Tags `lab-1` through `lab-8` preserve progressive snapshots.

The implementation and supporting artifacts are committed to the repository. This document summarizes the execution process and its evidence without reproducing the complete source code.

Verified result

44	89.94%	100%
passing tests	overall line coverage	service line / branch coverage

The final packaged Java application was started and checked over HTTP: five seed books, three seed members, health status UP, Swagger UI, and 14 documented operations. Static analysis and two deliberate fine-calculation bugs were also executed.

Roles and AI assistance

The student specified constraints, approved environment setup, reported a Maven failure, supplied the assignment, ran the application, and shared database/Swagger results. Codex performed the delegated implementation, test generation, automated analysis, and report preparation. The evidence distinguishes student interactions from agent-executed engineering activities.

> Execution evidence covers the Java application, automated tests, and a successful Docker image build and container HTTP check on GitHub's Linux runner. Docker was not installed on the local Windows host.

Interactions and decisions

The examples below are excerpts from the actual conversation. Agent responses are summarized unless explicitly presented as terminal output.

1. Specify scope and restrict changes

> “First inspect the workspace and verify the development environment. Then propose steps to configure Java 17 and install Maven.”
> “Do not modify files, install software, or change environment variables without my approval”

Agent action: read-only inspection found an empty Git repository, Java 24 selected in the process, working Temurin 17, and no Maven command. A setup proposal was returned before installation.

2. Approve concrete setup work

> “I approve the proposed environment setup.”
> “Download Apache Maven from the official Apache distribution, verify its SHA-512 checksum, and install it in my user directory.”

Agent action: installed Maven in the user directory, verified the SHA-512 digest, and updated user settings. System settings were not changed and Java 24 was retained.

3. Challenge an apparent success

> “mvn didnt work”

The supplied error said `mvn` was not recognized. The agent compared the saved user PATH with the terminal's inherited environment, then provided a session refresh. The student subsequently supplied output showing Maven 3.10.0 running on Java 17.0.21.

4. Guide the implementation with the handout

> “okay this is the deliverables, continue”

The initial skeleton was compared with the actual assignment. Missing entities, repositories, Book/Member routes, seed SQL, and Swagger were added. The student subsequently authorized completion of the remaining labs, which Codex implemented and checked through automated tests and analysis.

5. Validate observable behavior

The student ran `mvn spring-boot:run` and shared H2 and Swagger screenshots. The H2 view showed five book rows. Swagger showed `GET /books/1` returning 200 and an unknown ID returning 404 with `Book not found`. Codex then explained the controller-to-database request flow.

> Screenshot evidence is transcribed and identified as such in this report. A later read-only CI credential check required explicit permission after automatic review rejected the first attempt, providing another example of an enforced approval boundary.

Workflow and produced artifacts

The sequence followed inspection → proposal/authorization → edits → execution → review → refinement → versioned evidence.

Lab / SDLC activity	Outputs and observable evidence
1 • Setup and scaffold	Java 17 / Maven setup; three JPA entities; repositories; Book and Member CRUD; five books / three members; nine passing tests; lab-1 tag.
2 • Planning and TDD	Sprint backlog; borrowing, return, active loans and fines; red compile failure before service implementation; 15 passing tests; pair-review findings.
3 • Specification	docs/AGENTS.md with eight rules and five acceptance criteria; reservation service, routes and tests; 21 passing tests.
4 • Design and integration	DTOs/mappers, application services, shared error handler, annotated OpenAPI, multi-stage Dockerfile and CI workflow; 24 passing tests.
5 • Refactoring	Extract day computation; name fine/loan constants; centralize availability updates; three separate 24-test regression runs; before/after diff.
6 • Quality measurement	PMD, SpotBugs, JaCoCo; GQM plan; complexity/coupling tables; mutable error-map fix; 25 passing tests.
7 • Test evaluation	Mockito audit; real HTTP flows; fine mutations; 44 passing tests; final coverage 89.94% overall and 100% service line/branch.
8 • Estimation	79-UFP inventory; three-point size and COCOMO calculations; qualified LOC/FP analysis and measured agent-session time.

Representative artifacts

- docs/AGENTS.md — reservation requirements and acceptance criteria.
- docs/sprint-backlog.md — planning estimates and timing provenance.
- docs/lab5-refactor.diff — exact behavior-preserving changes.
- docs/gqm.md — measurement goal, questions, metrics, results.
- docs/lab7-test-audit.md and docs/lab7-mutations.json.
- Dockerfile and the verified CI workflow with container readiness checks.
- Eight detailed reports under reports/.

Architecture

HTTP JSON → DTO / controller → service → repository → H2

Controllers handle transport, services coordinate business rules and transactions, repositories provide persistence, and entities map stored data. DTOs keep the public contract independent of JPA relationship details.

Execution and test evidence

Student-supplied application startup

Extracted from the terminal output supplied in the session (11 November 2026, Asia/Beirut):

```
Java version: 17.0.21, vendor: Eclipse Adoptium
Apache Maven 3.10.0
Tomcat started on port 8080 (http)
Started LibraryApiApplication in 3.331 seconds
H2 console available at '/h2-console'
Database available at 'jdbc:h2:mem:librarydb'
```

Student-supplied endpoint evidence

Transcribed from the Swagger screenshots:

Request	Visible response
GET /books/1	200; id 1, title Clean Code, author Robert C. Martin, publishedYear 2008, available true.
GET /books/999999	404; title Not Found; detail Book not found; instance /books/999999.

Final automated test run

```
mvn --batch-mode --no-transfer-progress clean verify -Ddebug=false
Tests run: 44, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
Total time: 19.391 s
```

Suite (from Surefire XML)	Tests	Failures/errors
BorrowingHttpFlowTest	3	0/0
ErrorResponseTest	1	0/0
GlobalExceptionHandlerTest	3	0/0
ReservationControllerTest	1	0/0
LibraryApiApplicationTests	1	0/0
LibraryApiEndpointsTests	8	0/0
BorrowingServiceTest	6	0/0
BorrowingServiceUnitTest	16	0/0
ReservationServiceTest	5	0/0

The HTTP flow suite starts a real server on a random port and checks borrowing, a full return cycle, and duplicate-loan conflict. MockMvc checks cover validation, missing resources, CRUD, reservation criteria and OpenAPI. Mockito tests isolate BorrowingService without a Spring context.

Review, problems and refinement

Observed problem	Action and validation
Java 24 selected / Maven unavailable	Configured JDK 17, installed a verified Maven ZIP, and updated the user environment; the refreshed terminal reported Java 17 and Maven 3.10.0.
PowerShell download TLS decryption error	Retried using Windows curl with HTTPS verification enabled; SHA-512 matched before extraction.
Initial scaffold incomplete for the handout	Compared against specified fields/endpoints and added missing model, seed data, routes and documentation.
Inconsistent validation response shape	Made custom advice take precedence over Spring Boot's default problem-details advice; handler integration tests passed.
Mutable field-error map exposed	SpotBugs identified representation exposure; Map.copyOf plus an immutability test removed both targeted findings.
HTTP test client discarded generated IDs	Removed Jackson read-only ID binding from shared DTOs; retained OpenAPI read-only metadata and service-side ID protection; real HTTP tests passed.
CI queried the container before startup completed	The first request received a connection reset. A bounded health-readiness check now precedes API assertions; the Docker build and container checks subsequently passed.

Measured quality, with remaining findings

Metric	Before	After
Overall line coverage	75.79%	89.94%
Service line / branch coverage	100% / 94.44%	100% / 100%
SpotBugs findings	13	11 (targeted map warnings: 0)
PMD findings	15	6

Remaining analyzer findings were documented rather than suppressed to claim a clean report. JPA relationship-exposure warnings need interpretation; blindly copying managed entities would change persistence semantics.

Do the assertions detect wrong behavior?

After a passing baseline, the fine rate was temporarily changed from 0.50 to 1.00: **5 tests failed**. Returning null for an on-time fine caused **7 tests to fail**. Both original mutations were restored, and the final clean build passed. Exact failed/passed test names are preserved in the mutation evidence.

> Coverage indicates execution, not correctness. The deliberate bugs test whether specific assertions and database invariants protect the stated fine rules.

Reproducibility and final results

Execution and verification commands

The following commands were used during environment checks, application execution, and verification:

```
java -version
mvn -version
mvn clean verify
mvn spring-boot:run
mvn pmd:pmd spotbugs:spotbugs pmd:check
```

The environment used JDK 17 and Maven 3.10.0. The application ran on port 8080 during the student's checks; isolated automated smoke checks used port 18080. H2 provided an in-memory development database with reproducible seed records.

URL	Expected observation
http://localhost:8080/books	Five sample books on a fresh start
http://localhost:8080/members	Three sample members on a fresh start
http://localhost:8080/swagger-ui.html	Interactive API operations
http://localhost:8080/api-docs	OpenAPI JSON (path changed in Lab 4)
http://localhost:8080/actuator/health	status UP
http://localhost:8080/h2-console	JDBC jdbc:h2:mem:librarydb; user sa; blank password

Supporting evidence

The analysis commands generated PMD and SpotBugs XML reports. The test lifecycle generated JaCoCo HTML at `target/site/jacoco/index.html`. Committed XML snapshots and JSON summaries preserve the measured results without adding temporary build output to Git.

Test-first history: commit de6e233 contains borrowing tests before BorrowingService. Progressive tags are available on the repository's tags page. Detailed limitations: docs/submission-status.md.

Verification scope

The successful GitHub run verified Maven tests, the Docker image build, health status UP, five books, three members, and OpenAPI JSON. Containers ran on GitHub's Linux runner; local Windows execution used Java directly.

The results establish the documented functional and quality checks, while concurrent requests and production-database deployment remain outside the verified scope.

Conclusion

The Reference Lab produced a versioned REST API and a traceable sequence of specifications, tests, reviews, refactorings, and measurements. Environment, error-handling, and DTO failures prompted corrections backed by regression checks. This controlled exercise documents agentic development across SDLC activities and provides a reference for a separate final-project idea.